Ohjelmistotuotanto

Syksy 2026

Luento 1

26.10.2026

Ohjelmistotuotanto

Luennot:

- Matti Luukkainen
- Joulukuun vierailijat

Ohjaajat:

- Nea Kovalainen
- Tony Lam
- Jasse Merivirta
- Outi Savolainen
- Heidi Tapani
- Rasmus Viitanen

Team Ohtu



Ohjelmistotuotanto

?

Ohjelmistotuotanto

Johdanto *ohjelmistotuotantoon* (engl. software engineering), eli systemaattiseen tapaan tehdä hieman laajempia ohjelmistoja useamman hengen tiimissä ulkoiselle asiakkaalle

Ohjelmistotuotanto

Johdanto *ohjelmistotuotantoon* (engl. software engineering), eli systemaattiseen tapaan tehdä hieman laajempia ohjelmistoja useamman hengen tiimissä ulkoiselle asiakkaalle

Erityinen paino ns. ketterissä (engl. agile) ohjelmistotuotantomenetelmissä

Kurssin oppimistavoitteet

Valmiudet toimia *juniorikehittäjänä* pienessä tiimissä, esim. *ohjelmistoprojektissa* TKT20007

Kurssin oppimistavoitteet

Valmiudet toimia *juniorikehittäjänä* pienessä tiimissä, esim. *ohjelmistoprojektissa* TKT20007

Suoritettuaan kurssin opiskelija

- tuntee ohjelmistoprosessin vaiheet

Kurssin oppimistavoitteet

Valmiudet toimia *juniorikehittäjänä* pienessä tiimissä, esim. *ohjelmistoprojektissa* TKT20007

Suoritettuaan kurssin opiskelija

- tuntee ohjelmistoprosessin vaiheet
- tietää miten vaatimuksia hallitaan ketterässä kehityksessä

Kurssin oppimistavoitteet

Valmiudet toimia *juniorikehittäjänä* pienessä tiimissä, esim. *ohjelmistoprojektissa* TKT20007

Suoritettuaan kurssin opiskelija

- tuntee ohjelmistoprosessin vaiheet
- tietää miten vaatimuksia hallitaan ketterässä kehityksessä
- ymmärtää suunnittelun, toteutuksen ja testauksen roolin ketterässä kehityksessä

Kurssin oppimistavoitteet

Valmiudet toimia *juniorikehittäjänä* pienessä tiimissä, esim. *ohjelmistoprojektissa* TKT20007

Suoritettuaan kurssin opiskelija

- tuntee ohjelmistoprosessin vaiheet
- tietää miten vaatimuksia hallitaan ketterässä kehityksessä
- ymmärtää suunnittelun, toteutuksen ja testauksen roolin ketterässä kehityksessä
- ymmärtää laadunhallinnan perusteet

Kurssin oppimistavoitteet

Valmiudet toimia *juniorikehittäjänä* pienessä tiimissä, esim. *ohjelmistoprojektissa* TKT20007

Suoritettuaan kurssin opiskelija

- tuntee ohjelmistoprosessin vaiheet
- tietää miten vaatimuksia hallitaan ketterässä kehityksessä
- ymmärtää suunnittelun, toteutuksen ja testauksen roolin ketterässä kehityksessä
- ymmärtää laadunhallinnan perusteet
- osaa tehdä kehityksestä hallittua ja toistettavaa: versionhallinta, automatisoitu testaus, jatkuva integraatio

Kurssin oppimistavoitteet

Valmiudet toimia *juniorikehittäjänä* pienessä tiimissä, esim. *ohjelmistoprojektissa* TKT20007

Suoritettuaan kurssin opiskelija

- tuntee ohjelmistoprosessin vaiheet
- tietää miten vaatimuksia hallitaan ketterässä kehityksessä
- ymmärtää suunnittelun, toteutuksen ja testauksen roolin ketterässä kehityksessä
- ymmärtää laadunhallinnan perusteet
- osaa tehdä kehityksestä hallittua ja toistettavaa: versionhallinta, automatisoitu testaus, jatkuva integraatio
- osaa hyödyntää tekoälyä hallitusti sovelluskehityksessä

Esitietovaatimukset

- Tietokannat ja Web-ohjelmointi tai vastaavat tiedot

Esitietovaatimukset

- Tietokannat ja Web-ohjelmointi tai vastaavat tiedot
- Vanhalta nimeltään *Aineopintojen harjoitustyö: tietokantasovellus*

Esitietovaatimukset

- Tietokannat ja Web-ohjelmointi tai vastaavat tiedot
- Vanhalta nimeltään *Aineopintojen harjoitustyö: tietokantasovellus*
- ja näiden esitiedot: Ohpe, Ohja, Tikape
- hyötyä jos Ohte ja Lapio

Kurssin rakenne

- Luennot
  - ma ja ti 12-14
  - Ohjelmistotuotantoon liittyvää käsitteistöä ja teoriaa
  - Vierailuluennot (2 viimeistä viikkoa)

Kurssin rakenne

- Luennot
  - ma ja ti 12-14
  - Ohjelmistotuotantoon liittyvää käsitteistöä ja teoriaa
  - Vierailuluennot (2 viimeistä viikkoa)
- Laskarit
  - teoriaa kertaavat *monivalintatehtävät*, deadline su 23.59
  - versionhallintaa, testaamista ja ohjelmistojen konfigurointia käsittelevät, deadline *maanantaina klo 23.59*
    - viikon 7 deadline poikkeaa

Kurssin rakenne

- Luennot
  - ma ja ti 12-14
  - Ohjelmistotuotantoon liittyvää käsitteistöä ja teoriaa
  - Vierailuluennot (2 viimeistä viikkoa)
- Laskarit
  - teoriaa kertaavat *monivalintatehtävät*, deadline su 23.59
  - versionhallintaa, testaamista ja ohjelmistojen konfigurointia käsittelevät, deadline *maanantaina klo 23.59*
    - viikon 7 deadline poikkeaa
  - oletettu kuormittavuus: viikoilla 1–2 noin 8 tuntia, viikolla 3 noin 6 tuntia, viikoilla 4–5 noin 4 tuntia ja viikolla 6 noin 6 tuntia
  - monivalintoihin vastaaminen nopeaa, jos...

Kurssin rakenne

- Luennot
  - ma ja ti 12-14
  - Ohjelmistotuotantoon liittyvää käsitteistöä ja teoriaa
  - Vierailuluennot (2 viimeistä viikkoa)
- Laskarit
  - teoriaa kertaavat *monivalintatehtävät*, deadline su 23.59
  - versionhallintaa, testaamista ja ohjelmistojen konfigurointia käsittelevät, deadline *maanantaina klo 23.59*
    - viikon 7 deadline poikkeaa
  - oletettu kuormittavuus: viikoilla 1–2 noin 8 tuntia, viikolla 3 noin 6 tuntia, viikoilla 4–5 noin 4 tuntia ja viikolla 6 noin 6 tuntia
  - monivalintoihin vastaaminen nopeaa, jos...
- Miniprojekti
  - alkaa kurssin 3. viikolla
  - yhdistää teorian ja käytännön

Miniprojekti

- kurssin viikoilla 3–7
- ryhmätyö: koodataan hiukan, harjoitellaan projektinhallintaa sekä eräitä laadunhallintatekniikoita

Miniprojekti

- kurssin viikoilla 3–7
- ryhmätyö: koodataan hiukan, harjoitellaan projektinhallintaa sekä eräitä laadunhallintatekniikoita
- ryhmässä 4–6 opiskelijaa, ryhmillä on myös asiakas, jota tavataan viikoittain
- ensimmäisellä viikolla asiakastapaamiseen tulee varata 90 minuuttia, myöhemmillä viikoilla 30 minuuttia
- kurssin lopussa on miniprojektien yhteinen kahden tunnin mittainen demotilaisuus

Miniprojekti

- kurssin viikoilla 3–7
- ryhmätyö: koodataan hiukan, harjoitellaan projektinhallintaa sekä eräitä laadunhallintatekniikoita
- ryhmässä 4–6 opiskelijaa, ryhmillä on myös asiakas, jota tavataan viikoittain
- ensimmäisellä viikolla asiakastapaamiseen tulee varata 90 minuuttia, myöhemmillä viikoilla 30 minuuttia
- kurssin lopussa on miniprojektien yhteinen kahden tunnin mittainen demotilaisuus
- miniprojekteissa työskentelyyn tulee varata aikaa noin 6 tuntia viikossa

Kurssin läpäisyn edellytyksenä on hyväksytysti suoritettu tai hyväksiluettu miniprojekti

Miniprojektit

- Käynnistyvät 9.11. alkavalla viikolla
- **Ilmoittautumisen deadline la 7.11. klo 23.59**
- Aloitustilaisuudet
  - ma 14-16
  - to 14-16
  - ke 10-12
  - ke 12-14
  - to 14-16
  - to 16-18
- Seuraavat viikot: sprinttien katselmus ja suunnittelu samassa aikaikkunassa
- Loppudemot
  - ke 9.12. klo 10-12 B123
  - to 10.12. klo 10-12 A111

Miniprojektin hyväksilukeminen

Vähintään kolmen kuukauden työkokemus tiimityönä tehtävästä ohjelmistokehityksestä

Lähetä sähköpostia sen jälkeen, kun olet palauttanut viikon 1 tehtävät

Kurssin arvostelu

Jaossa yhteensä 40 pistettä

- laskarit 10 pistettä
  - monivalintatehtävät 2 pistettä
  - viikoittaiset ohjelmointi/versionhallinta/konfigurointitehtävät 8 pistettä
- miniprojekti 11 pistettä
- koe 18 pistettä
- osallistuminen vierailuluennoille 1 piste (0,33 pistettä per luentokerta, täydet pisteet kolmesta kerrasta)

Arvosanaan 1 riittää 20 pistettä, arvosanaan 5 tarvitaan 36 pistettä.

Läpipääsy edellyttää lisäksi miniprojektin hyväksyttyä suoritusta (tai hyväksilukua) ja vähintään puolet kokeen pisteistä

Luennot - laskarit - miniprojekti

Luennoilla ohjelmistokehityksen teoriaa ja käsitteistöä

- laskarien *monivalintatehtävät* liittyvät kunkin viikon luentoihin

Luennot - laskarit - miniprojekti

Luennoilla ohjelmistokehityksen teoriaa ja käsitteistöä

- laskarien *monivalintatehtävät* liittyvät kunkin viikon luentoihin

Versionhallintaa, konfigurointia, testausta ja ohjelmointia käsittelevien **teknisempien laskarien** aihepiirejä ei paljoa käsitellä luennoilla

Luennot - laskarit - miniprojekti

Luennoilla ohjelmistokehityksen teoriaa ja käsitteistöä

- laskarien *monivalintatehtävät* liittyvät kunkin viikon luentoihin

Versionhallintaa, konfigurointia, testausta ja ohjelmointia käsittelevien **teknisempien laskarien** aihepiirejä ei paljoa käsitellä luennoilla

Miniprojekti yhdistää luentojen teorian ja laskareissa käsitellyt teknisemmät asiat, ja soveltaa niitä käytännössä

Luennot - laskarit - miniprojekti

Luennoilla ohjelmistokehityksen teoriaa ja käsitteistöä

- laskarien *monivalintatehtävät* liittyvät kunkin viikon luentoihin

Versionhallintaa, konfigurointia, testausta ja ohjelmointia käsittelevien **teknisempien laskarien** aihepiirejä ei paljoa käsitellä luennoilla

Miniprojekti yhdistää luentojen teorian ja laskareissa käsitellyt teknisemmät asiat, ja soveltaa niitä käytännössä

Kokeessa suurin painoarvo teoriassa ja sen soveltamisessa käytäntöön

- laskareiden teknisimpiä asioita ei kokeessa kysytä
- tarkemmin kokeesta kurssin viimeisellä luennolla

Versionhallinta 1 op

Saat suoritusmerkinnän tekemällä kaikki kurssin versionhallintatehtävät ja suorittamalla hyväksytysti miniprojektin (tai hyväksilukemalla sen)

Kurssimateriaali

https://ohjelmistotuotanto-hy.github.io/

Kurssipalaute

Kurssipalaute

- Kurssipalaute
  - Kurssin lopussa kerättävän palautteen lisäksi ns. jatkuva palaute https://norppa.helsinki.fi

Kurssipalaute

- Kurssipalaute
  - Kurssin lopussa kerättävän palautteen lisäksi ns. jatkuva palaute https://norppa.helsinki.fi
- Kurssipalaute **vaikuttaa**
- Edellisen kurssin palautteen “moitteita”
  - Miniprojekti
    - Ryhmäjako voisi ottaa huomioon sen, haluaako ryhmä työskennellä ensisijaisesti kampuksella vai etänä
    - Alkuun enemmän ohjeita ja aikaa ryhmätyötaitojen kehittämiseen ja sovelluksen infrastruktuurin pystyttämiseen

Monivalintojen korvaaminen luentopäiväkirjalla

Monivalintojen korvaaminen luentopäiväkirjalla

- Kurssin viikoilla 1–5 viikon luentoja kertaavat monivalinnat (2/40 kurssin pisteistä)

Monivalintojen korvaaminen luentopäiväkirjalla

- Kurssin viikoilla 1–5 viikon luentoja kertaavat monivalinnat (2/40 kurssin pisteistä)
- Monivalinnat herättäneet tunteita, en ole itsekään niiden fani
- Hyödyt ovat kiistattomat, kokeet menevät nykyään paremmin kuin monivalintoja edeltävänä aikana
  - linjassa ns. testausvaikutuksen (testing effect) kanssa: asioiden mieleen palauttaminen tehostaa oppimista enemmän kuin uudelleenlukeminen

Monivalintojen korvaaminen luentopäiväkirjalla

- Kurssin viikoilla 1–5 viikon luentoja kertaavat monivalinnat (2/40 kurssin pisteistä)
- Monivalinnat herättäneet tunteita, en ole itsekään niiden fani
- Hyödyt ovat kiistattomat, kokeet menevät nykyään paremmin kuin monivalintoja edeltävänä aikana
  - linjassa ns. testausvaikutuksen (testing effect) kanssa: asioiden mieleen palauttaminen tehostaa oppimista enemmän kuin uudelleenlukeminen
- Tarjolla vaihtoehto: **viikoittainen luentopäiväkirja**
- Noin A4:n kokoinen omin sanoin tehty yhteenveto viikon luentojen asioista
  - voi olla tehty käsin, joko tekstiä, lista ranskalaisia viivoja tai esim. mind map
  - ei kuitenkaan missään tapauksessa plagiaatti tai tekoälyllä generoitu (opintovilppi: käsitellään HY:n prosessin mukaan)

Monivalintojen korvaaminen luentopäiväkirjalla

- Kurssin viikoilla 1–5 viikon luentoja kertaavat monivalinnat (2/40 kurssin pisteistä)
- Monivalinnat herättäneet tunteita, en ole itsekään niiden fani
- Hyödyt ovat kiistattomat, kokeet menevät nykyään paremmin kuin monivalintoja edeltävänä aikana
  - linjassa ns. testausvaikutuksen (testing effect) kanssa: asioiden mieleen palauttaminen tehostaa oppimista enemmän kuin uudelleenlukeminen
- Tarjolla vaihtoehto: **viikoittainen luentopäiväkirja**
- Noin A4:n kokoinen omin sanoin tehty yhteenveto viikon luentojen asioista
  - voi olla tehty käsin, joko tekstiä, lista ranskalaisia viivoja tai esim. mind map
  - ei kuitenkaan missään tapauksessa plagiaatti tai tekoälyllä generoitu (opintovilppi: käsitellään HY:n prosessin mukaan)
- Deadline kunkin viikon sunnuntai klo 23.59, lähetetään sähköpostitse matti.luukkainen@helsinki.fi

Paja

- BK107
  - ma 14.30-16.30
  - ti 12-16
  - to 12-16
  - pe 12-14

Ohjelmistotuotanto engl. software engineering

Ohjelmistotuotanto engl. software engineering

The IEEE Computer Society defines software engineering as:

- *The application of a systematic, disciplined, quantifiable approach to the development, operation, and maintenance of software; that is, the application of engineering to software*

Ohjelmistotuotanto engl. software engineering

The IEEE Computer Society määritelmän mukaan

- *Systemaattisen, kurinalaisen ja mitattavissa olevan lähestymistavan soveltaminen ohjelmistojen kehittämiseen, käyttöön ja ylläpitoon, eli "insinöörityön" soveltaminen ohjelmistoihin*

Ohjelmistotuotanto engl. software engineering

The IEEE Computer Society määritelmän mukaan

- *Systemaattisen, kurinalaisen ja mitattavissa olevan lähestymistavan soveltaminen ohjelmistojen kehittämiseen, käyttöön ja ylläpitoon, eli "insinöörityön" soveltaminen ohjelmistoihin*

Alun perin IEEE:n terminologiastandardista (IEEE Std 610.12-1990, nykyään ISO/IEC/IEEE 24765), käytössä myös *SWEBOK*:issa eli *Guide to the Software Engineering Body of Knowledge*:ssa

- ison komitean yritys määritellä mitä ohjelmistotuotannolla tarkoitetaan ja mitä osa-alueita siihen kuuluu
- uusin versio 4 ilmestyi vuonna 2024

Ohjelmistotuotannon osa-alueet SWEBOK:in mukaan

- Software requirements eli **vaatimusmäärittely**
- Software design eli **suunnittelu**
- Software construction eli **toteutus/ohjelmointi**
- Software testing eli **testaus**
- Software maintenance eli **ylläpito**

Ohjelmistotuotannon osa-alueet SWEBOK:in mukaan

- Software requirements eli **vaatimusmäärittely**
- Software design eli **suunnittelu**
- Software construction eli **toteutus/ohjelmointi**
- Software testing eli **testaus**
- Software maintenance eli **ylläpito**

- Software configuration management eli **konfiguraatiot**
- Software engineering models and methods eli **mallit ja menetelmät**
- Software quality eli **laadunhallinta**
- Software engineering process eli **ohjelmistotuotantoprosessi**
- Software engineering management eli **projektien johtaminen**

Ohjelmistotuotannon osa-alueet SWEBOK:in mukaan

- Software requirements eli **vaatimusmäärittely**
- Software design eli **suunnittelu**
- Software construction eli **toteutus/ohjelmointi**
- Software testing eli **testaus**
- Software maintenance eli **ylläpito**

- Software configuration management eli **konfiguraatiot**
- Software engineering models and methods eli **mallit ja menetelmät**
- Software quality eli **laadunhallinta**
- Software engineering process eli **ohjelmistotuotantoprosessi**
- Software engineering management eli **projektien johtaminen**

- Software architecture eli **arkkitehtuuri**
- Software engineering operations eli **operointi tuotannossa**
- Software security eli **tietoturva**
- Software engineering economics eli **talous**
- Software engineering professional practice eli **ammattietiikka ja ryhmädynamiikka**

Ohjelmiston elinkaari (software lifecycle)

Riippumatta tyylistä ja tavasta, jolla ohjelmisto tehdään, käy ohjelmisto läpi seuraavat *vaiheet*

- Vaatimusten analysointi ja määrittely
- Suunnittelu
- Toteutus
- Testaus
- Ohjelmiston ylläpito ja evoluutio

Ohjelmiston elinkaari (software lifecycle)

Riippumatta tyylistä ja tavasta, jolla ohjelmisto tehdään, käy ohjelmisto läpi seuraavat *vaiheet*

- Vaatimusten analysointi ja määrittely
- Suunnittelu
- Toteutus
- Testaus
- Ohjelmiston ylläpito ja evoluutio

Vaiheista muodostuu ohjelmiston “elinkaari”

Ohjelmiston elinkaari (software lifecycle)

Riippumatta tyylistä ja tavasta, jolla ohjelmisto tehdään, käy ohjelmisto läpi seuraavat *vaiheet*

- Vaatimusten analysointi ja määrittely
- Suunnittelu
- Toteutus
- Testaus
- Ohjelmiston ylläpito ja evoluutio

Vaiheista muodostuu ohjelmiston “elinkaari”

Käytetty **ohjelmistotuotantoprosessi** määrittelee miten vaiheet suhtautuvat toisiinsa

Ohjelmiston elinkaari (software lifecycle)

Riippumatta tyylistä ja tavasta, jolla ohjelmisto tehdään, käy ohjelmisto läpi seuraavat *vaiheet*

- Vaatimusten analysointi ja määrittely
- Suunnittelu
- Toteutus
- Testaus
- Ohjelmiston ylläpito ja evoluutio

Vaiheista muodostuu ohjelmiston “elinkaari”

Käytetty **ohjelmistotuotantoprosessi** määrittelee miten vaiheet suhtautuvat toisiinsa

Eri vaiheiden sisältöön palaamme myöhemmin tarkemmin

Alussa (ja osin edelleen) code'n'fix

Historian alkuaikoina laitteet maksoivat paljon, ohjelmat olivat laitteistoihin nähden "triviaaleja"

- ohjelmointi konekielellä
- sovelluksen käyttäjä ohjelmoi itse ohjelmansa

Alussa (ja osin edelleen) code'n'fix

Historian alkuaikoina laitteet maksoivat paljon, ohjelmat olivat laitteistoihin nähden "triviaaleja"

- ohjelmointi konekielellä
- sovelluksen käyttäjä ohjelmoi itse ohjelmansa

Vähitellen ohjelmistot alkavat kasvaa ja kehitettiin korkeamman tason ohjelmointikieliä (Fortran, Cobol, Algol)

- sovellusalue laajenee monille elämänaloille

Alussa (ja osin edelleen) code'n'fix

Historian alkuaikoina laitteet maksoivat paljon, ohjelmat olivat laitteistoihin nähden "triviaaleja"

- ohjelmointi konekielellä
- sovelluksen käyttäjä ohjelmoi itse ohjelmansa

Vähitellen ohjelmistot alkavat kasvaa ja kehitettiin korkeamman tason ohjelmointikieliä (Fortran, Cobol, Algol)

- sovellusalue laajenee monille elämänaloille

Pikkuhiljaa homma alkaa karata käsistä:

- budjetit ylittyivät ja projektit myöhästyivät aikatauluista
- ohjelmistot olivat tehottomia, niiden laatu oli huono ja ne eivät toimineet käyttäjien tarpeiden mukaan
- koodin ylläpito ja laajentaminen oli vaikeaa
- usein ohjelmistoja ei hyvistä aikeista huolimatta saatu ollenkaan toimitettua

Ohjelmistokriisi

Termi Software crisis lanseerataan lokakuussa 1968 NATO:n konferenssissa Garmischissa

- In essence, it refers to the difficulty of writing correct, understandable, and verifiable computer programs.

Ohjelmistokriisi

Termi Software crisis lanseerataan lokakuussa 1968 NATO:n konferenssissa Garmischissa

- In essence, it refers to the difficulty of writing correct, understandable, and verifiable computer programs.

Edsger Dijkstra:

- **as long as there were no machines, programming was no problem at all**

Ohjelmistokriisi

Termi Software crisis lanseerataan lokakuussa 1968 NATO:n konferenssissa Garmischissa

- In essence, it refers to the difficulty of writing correct, understandable, and verifiable computer programs.

Edsger Dijkstra:

- **as long as there were no machines, programming was no problem at all**
- when we had a few weak computers, programming became a mild problem

Ohjelmistokriisi

Termi Software crisis lanseerataan lokakuussa 1968 NATO:n konferenssissa Garmischissa

- In essence, it refers to the difficulty of writing correct, understandable, and verifiable computer programs.

Edsger Dijkstra:

- **as long as there were no machines, programming was no problem at all**
- when we had a few weak computers, programming became a mild problem
- **now we have gigantic computers, programming has become an equally gigantic problem**.

Kriisi ei ole ohi (syksy 2023)

Kaupunki | Koulut

Helsingiltä paljastui taas massiivinen epäonnistuminen: Uusi järjestelmä on susi

Koulujen ja päiväkotien ASTI-järjestelmällä piti korvata lopulta esimerkiksi Wilma, mutta 32 miljoonaa syönyt tietojärjestelmä jää keskentekoiseksi.



Helsingin kaupungintalo. KUVA: SAARA MANSIKKAMÄKI / HS

DIGITODAY

91 miljoonan euron ja 14 vuoden fiasko: Poliisin tietojärjestelmä-hanke kaatui

Hanke käynnistettiin vuonna 2009.

JAA TALLENNA KOMMENTIT



Software development as Engineering

Software development as Engineering

Termi **software engineering** nousee laajaan tietoisuuteen 1968 NATO:n konferenssissa (käytössä jo aiemmin, mm. Margaret Hamilton). Fritz Bauerin määritelmä:

- *The establishment and use of sound engineering principles in order to obtain economically software that is reliable and works efficiently on real machines*

Software development as Engineering

Termi **software engineering** nousee laajaan tietoisuuteen 1968 NATO:n konferenssissa (käytössä jo aiemmin, mm. Margaret Hamilton). Fritz Bauerin määritelmä:

- *The establishment and use of sound engineering principles in order to obtain economically software that is reliable and works efficiently on real machines*

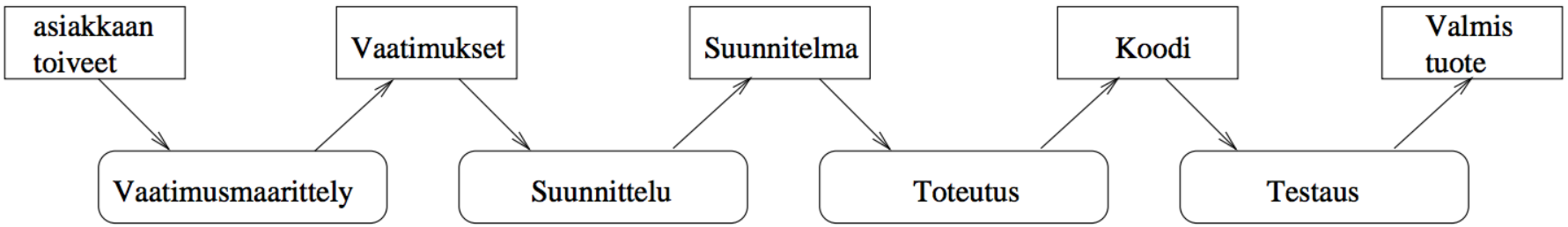
Ajatus siitä, että ohjelmistojen tekemisen tulisi olla kuin mikä tahansa muu insinöörityö

- ensin rakennettava artefakti *määritellään* (requirements)
- ja *suunnitellaan* (design) aukottomasti
- tämän jälkeen *rakentaminen* (construction) on melko suoraviivainen vaihe

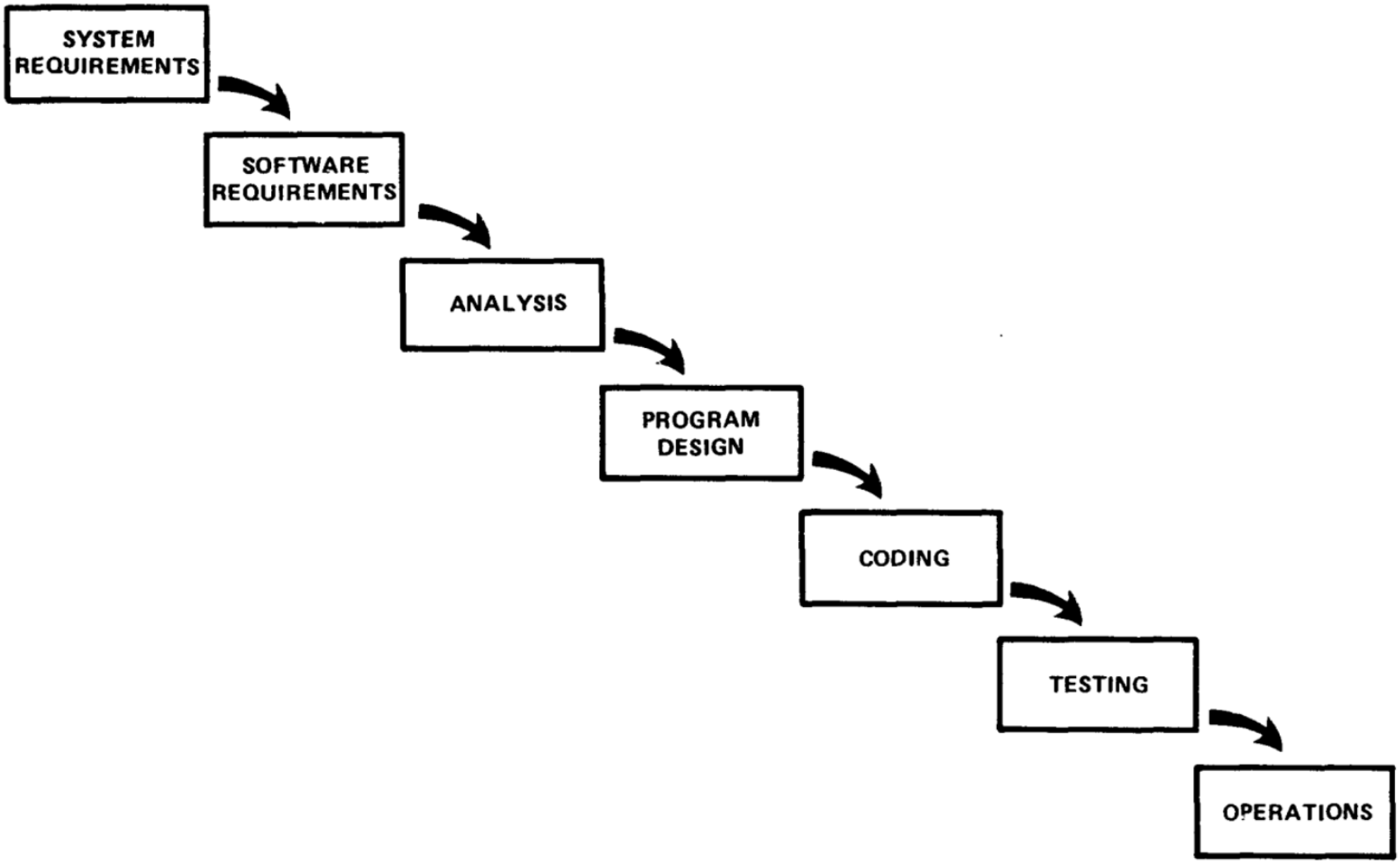
Vesiputousmalli

Winston W. Royce: Managing the Development of Large Software Systems, 1970

Sivulla 2 Royce esittelee yksinkertaisen prosessimallin, jossa elinkaaren vaiheet suoritetaan lineaarisesti peräkkäin:



Vesiputousmalli Roycen artikkelista



Vesiputousmallin suosion taustaa

Vesiputousmalli saavutti nopeasti suosiota

Yhdysvaltain puolustusministeriö rupesi vaatimaan kaikilta alihankkijoiltaan vesiputousmallin noudattamista

- Standardit DOD-STD-2167 (1985) ja DOD-STD-2167A (1988)

Vesiputousmallin suosion taustaa

Vesiputousmalli saavutti nopeasti suosiota

Yhdysvaltain puolustusministeriö rupesi vaatimaan kaikilta alihankkijoiltaan vesiputousmallin noudattamista

- Standardit DOD-STD-2167 (1985) ja DOD-STD-2167A (1988)

Muutkin ohjelmistoja tuottaneet tahot ajattelivat, että koska DoD vaatii vesiputousmallia, tapa kannattaa omaksua itselleen

Vesiputousmallin oletuksia

Vesiputousmalli perustuu vahvasti siihen, että *eri vaiheet ovat erillisten tuotantotiimien tekemiä*

- Tämän takia kunkin vaiheen tulokset *dokumentoidaan tarkoin*

Vesiputousmallin oletuksia

Vesiputousmalli perustuu vahvasti siihen, että *eri vaiheet ovat erillisten tuotantotiimien tekemiä*

- Tämän takia kunkin vaiheen tulokset *dokumentoidaan tarkoin*

Vaiheet tehdään peräkkäin

- esim. tekninen suunnittelu aloitetaan kun vaatimusmäärittely on valmis

Vesiputousmallin oletuksia

Vesiputousmalli perustuu vahvasti siihen, että *eri vaiheet ovat erillisten tuotantotiimien tekemiä*

- Tämän takia kunkin vaiheen tulokset *dokumentoidaan tarkoin*

Vaiheet tehdään peräkkäin

- esim. tekninen suunnittelu aloitetaan kun vaatimusmäärittely on valmis

Vesiputousmallin mukainen ohjelmistoprosessi on yleensä etukäteen *tarkkaan suunniteltu, resursoitu ja aikataulutettu*

Vesiputousmallin oletuksia

Vesiputousmalli perustuu vahvasti siihen, että *eri vaiheet ovat erillisten tuotantotiimien tekemiä*

- Tämän takia kunkin vaiheen tulokset *dokumentoidaan tarkoin*

Vaiheet tehdään peräkkäin

- esim. tekninen suunnittelu aloitetaan kun vaatimusmäärittely on valmis

Vesiputousmallin mukainen ohjelmistoprosessi on yleensä etukäteen *tarkkaan suunniteltu, resursoitu ja aikataulutettu*

Vesiputousmallin mukainen ohjelmistotuotanto ei ole osoittautunut erityisen onnistuneeksi

Vesiputousmallin ongelmia

Asiakkaan *vaatimukset muuttuvat* usein matkan varrella:

- Asiakas ei tiedä tai osaa sanoa mitä haluaa/tarvitsee
- Asiakkaan tarve muuttuu projektin kuluessa
- Asiakas alkaa haluta muutoksia kun näkee lopputuotteen

Vesiputousmallin ongelmia

Asiakkaan *vaatimukset muuttuvat* usein matkan varrella:

- Asiakas ei tiedä tai osaa sanoa mitä haluaa/tarvitsee
- Asiakkaan tarve muuttuu projektin kuluessa
- Asiakas alkaa haluta muutoksia kun näkee lopputuotteen

Vaatimusmäärittelyn, suunnittelun ja toteutuksen erottaminen ei järkevää

- Ohjelmaa on mahdotonta suunnitella siten, että toteutus on suoraviivaista
- Toteutusteknologiat vaikuttavat suuresti määriteltyjen ominaisuuksien hintaan
- Osa suunnittelusta tapahtuu pakosti vasta ohjelmoitaessa

Vesiputousmallin ongelmia

Asiakkaan *vaatimukset muuttuvat* usein matkan varrella:

- Asiakas ei tiedä tai osaa sanoa mitä haluaa/tarvitsee
- Asiakkaan tarve muuttuu projektin kuluessa
- Asiakas alkaa haluta muutoksia kun näkee lopputuotteen

Vaatimusmäärittelyn, suunnittelun ja toteutuksen erottaminen ei järkevää

- Ohjelmaa on mahdotonta suunnitella siten, että toteutus on suoraviivaista
- Toteutusteknologiat vaikuttavat suuresti määriteltyjen ominaisuuksien hintaan
- Osa suunnittelusta tapahtuu pakosti vasta ohjelmoitaessa

Lopuksi tapahtuva laadunhallinta paljastaa ongelmat liian myöhään

- Korjaukset mahdollisesti kalliita: voi paljastua ongelmia jotka pakottavat muuttamaan ohjelmiston vaatimuksia

Vesiputous oli väärinymmärrys

Paradoksaalista kyllä vesiputousmallin isänä pidetty Royce **ei suosittele** artikkelissaan suoraviivaisen lineaarisen mallin käyttöä

Vesiputous oli väärinymmärrys

Paradoksaalista kyllä vesiputousmallin isänä pidetty Royce **ei suosittele** artikkelissaan suoraviivaisen lineaarisen mallin käyttöä

Royce esittelee lineaarisen vesiputousmallin **sivulla 2**, mutta toteaa että se **ei sovellu** monimutkaisiin ohjelmistoprojekteihin

Vesiputous oli väärinymmärrys

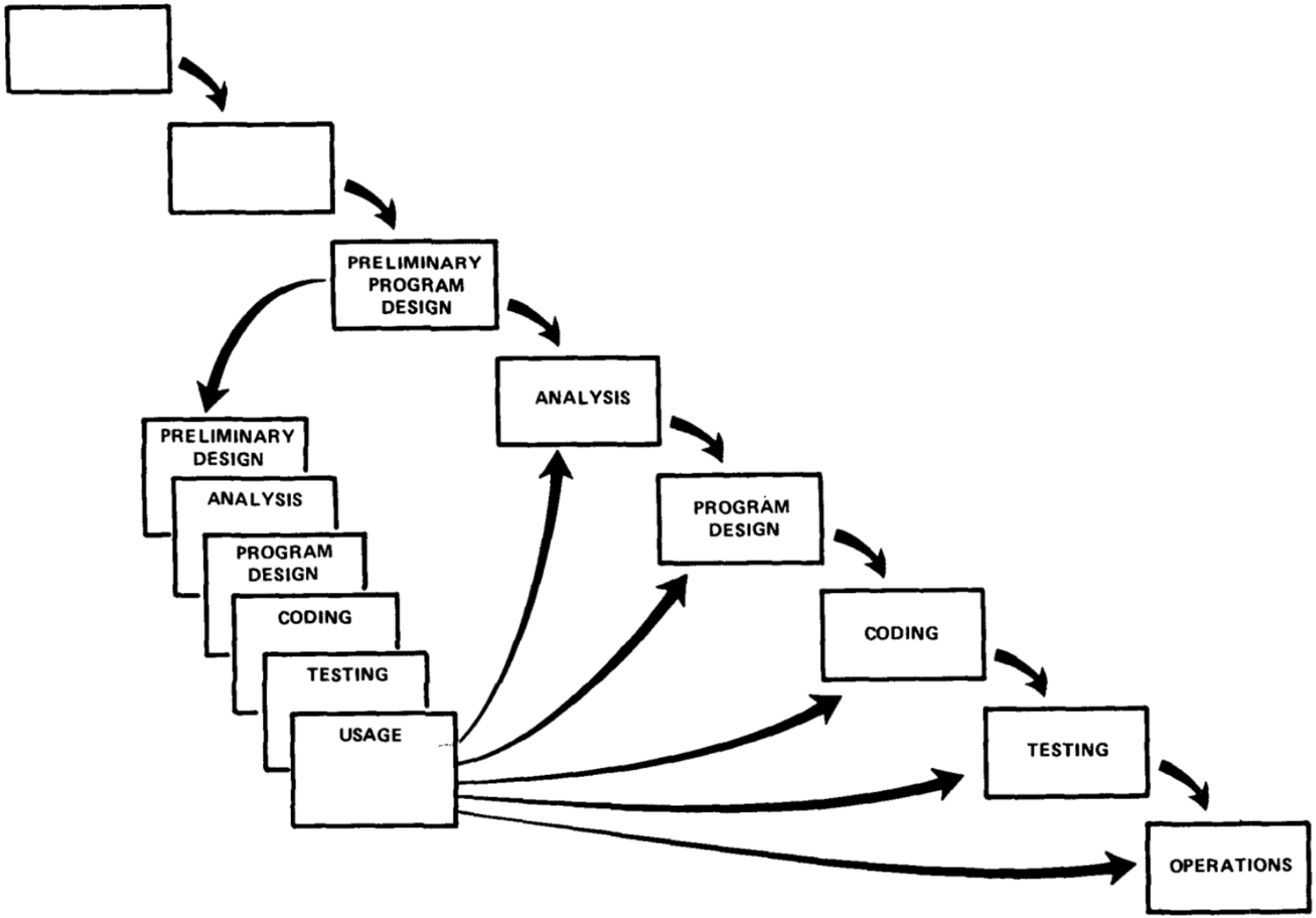
Paradoksaalista kyllä vesiputousmallin isänä pidetty Royce **ei suosittele** artikkelissaan suoraviivaisen lineaarisen mallin käyttöä

Royce esittelee lineaarisen vesiputousmallin **sivulla 2**, mutta toteaa että se **ei sovellu** monimutkaisiin ohjelmistoprojekteihin

Roycen mukaan

- ensin tulee tehdä prototyyppi
- ja siitä saatujen kokemusten valossa suunnitellaan ja toteutetaan lopullinen ohjelmisto

Roycen kahden iteraation malli



Iteratiivinen ohjelmistokehitys

Vesiputousmallin ongelmiin reagoinut *iteratiivinen* tapa tehdä ohjelmistoja alkoi yleistyä 80- ja 90-luvuilla

- mm. spiraalimalli, prototyyppimalli, Rational Unified Process

Iteratiivinen ohjelmistokehitys

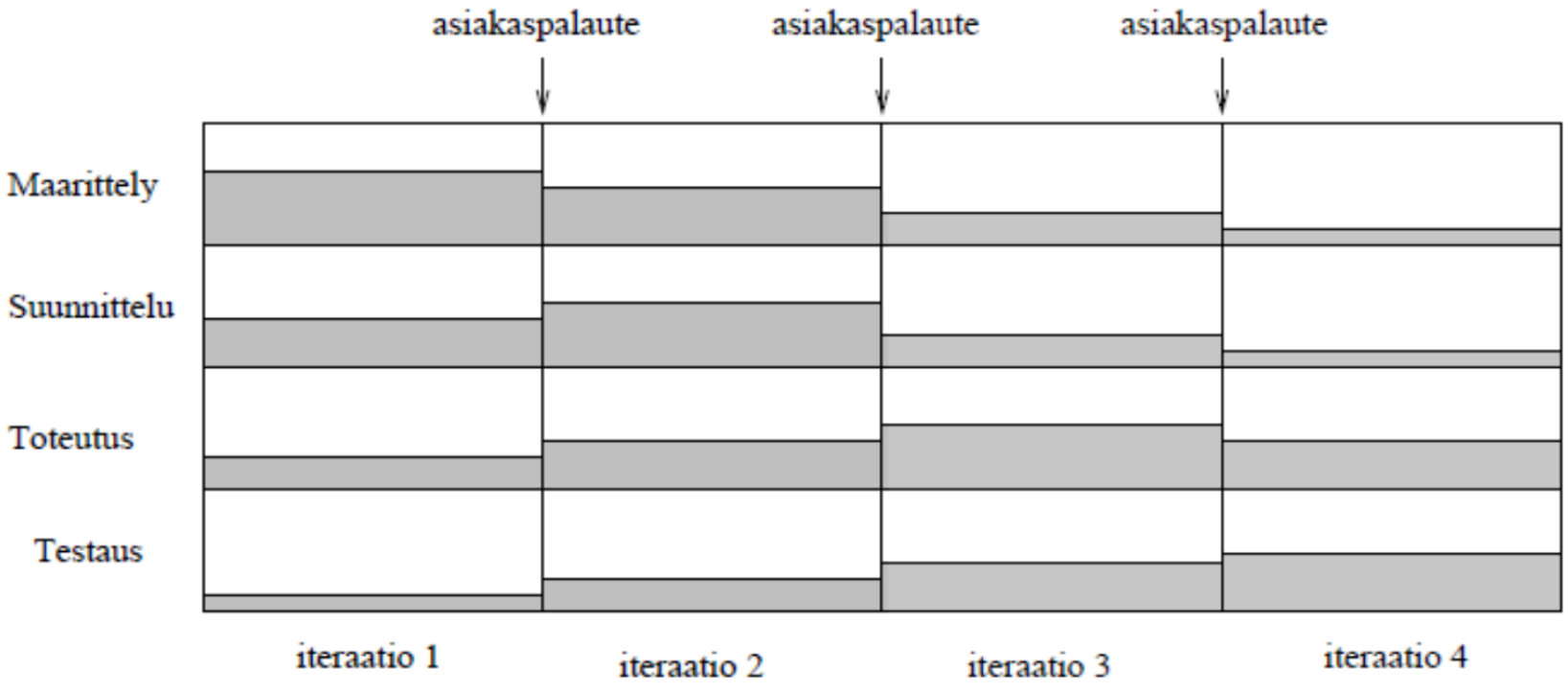
Vesiputousmallin ongelmiin reagoinut *iteratiivinen* tapa tehdä ohjelmistoja alkoi yleistyä 80- ja 90-luvuilla

- mm. spiraalimalli, prototyyppimalli, Rational Unified Process

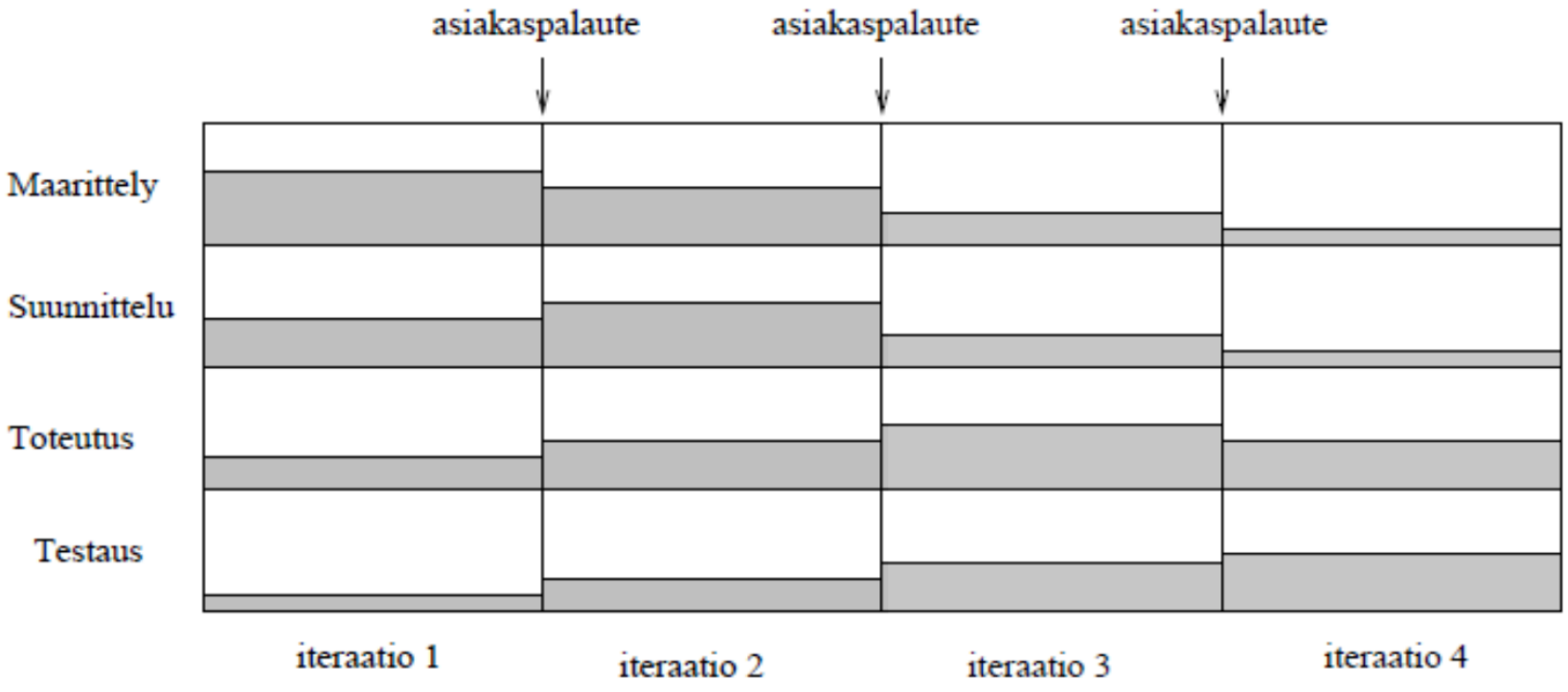
Iteratiivisessa ohjelmistokehityksessä

- ohjelmistotuotanto jaetaan jaksoihin, eli *iteraatioihin*
- jokaisen iteraation aikana määritellään, suunnitellaan, toteutetaan ja testataan ohjelmistoa
- eli ohjelmisto kehittyy vähitellen (inkrementaalisesti)

Iteratiivinen ohjelmistokehitys



Iteratiivinen ohjelmistokehitys



- asiakasta tavataan jokaisen iteraation välissä
- asiakas näkee sen hetkisen version ohjelmasta ja pystyy vaikuttamaan seuraavien iteraatioiden kulkuun

Iteratiivinen ohjelmistokehitys ei ole uusi keksintö

Yhdysvaltojen puolustusministeriön vuonna 2000 julkaisema hankintaohje DoD 5000.2 alkaa suositella iteratiivista ohjelmistoprosessia:

- There are two approaches, evolutionary (*iterative*) and single step (*waterfall*), to full capability. An *evolutionary approach is preferred*...

Iteratiivinen ohjelmistokehitys ei ole uusi keksintö

Yhdysvaltojen puolustusministeriön vuonna 2000 julkaisema hankintaohje DoD 5000.2 alkaa suositella iteratiivista ohjelmistoprosessia:

- There are two approaches, evolutionary (*iterative*) and single step (*waterfall*), to full capability. An *evolutionary approach is preferred*...

Iteratiivinen ohjelmistokehitys on paljon vanhempi idea kuin vesiputousmalli

- esim. NASA:n ensimmäisen amerikkalaisen avaruuteen vieneen Project Mercuryn ohjelmisto kehitettiin 50- ja 60-lukujen taitteessa iteratiivisesti
- avaruussukkuloiden ohjelmisto tehtiin vesiputousmallin valtakaudella, mutta iteratiivisesti
  - 8 viikon iteraatioissa, 31 kuukaudessa

Iteratiivinen ohjelmistokehitys ei ole uusi keksintö

Yhdysvaltojen puolustusministeriön vuonna 2000 julkaisema hankintaohje DoD 5000.2 alkaa suositella iteratiivista ohjelmistoprosessia:

- There are two approaches, evolutionary (*iterative*) and single step (*waterfall*), to full capability. An *evolutionary approach is preferred*...

Iteratiivinen ohjelmistokehitys on paljon vanhempi idea kuin vesiputousmalli

- esim. NASA:n ensimmäisen amerikkalaisen avaruuteen vieneen Project Mercuryn ohjelmisto kehitettiin 50- ja 60-lukujen taitteessa iteratiivisesti
- avaruussukkuloiden ohjelmisto tehtiin vesiputousmallin valtakaudella, mutta iteratiivisesti
  - 8 viikon iteraatioissa, 31 kuukaudessa

Roycen artikkelikin (vuonna 1970) ehdotti *kahden iteraation* menetelmää ohjelmistojen tekemiseen

Ketterien menetelmien synty...

Perinteisissä prosessimalleissa korostettiin

- huolellista projektisuunnittelua
- tarkkaa dokumentaatiota
- yksityiskohtaisia analyysi- ja suunnittelumenetelmiä
- täsmällistä, tarkasti ohjattua ohjelmistoprosessia

Ketterien menetelmien synty...

Perinteisissä prosessimalleissa korostettiin

- huolellista projektisuunnittelua
- tarkkaa dokumentaatiota
- yksityiskohtaisia analyysi- ja suunnittelumenetelmiä
- täsmällistä, tarkasti ohjattua ohjelmistoprosessia

Tukivat erityisesti laajojen, pitkäikäisten ohjelmistojen kehitystyötä

- pienten ja keskisuurten ohjelmistojen tekoon turhan jäykkiä

Ketterien menetelmien synty...

Perinteisissä prosessimalleissa korostettiin

- huolellista projektisuunnittelua
- tarkkaa dokumentaatiota
- yksityiskohtaisia analyysi- ja suunnittelumenetelmiä
- täsmällistä, tarkasti ohjattua ohjelmistoprosessia

Tukivat erityisesti laajojen, pitkäikäisten ohjelmistojen kehitystyötä

- pienten ja keskisuurten ohjelmistojen tekoon turhan jäykkiä

Pyrittiin työtä tekevän yksilön merkityksen minimoimiseen

- yksilö on “tehdastyöläinen”, joka voidaan helposti korvata toisella ja tällä ei ole ohjelmiston kehittämiseen vaikutusta

Ketterien menetelmien synty

Ristiriita/turhauma synnytti joukon **ketteriä prosessimalleja** (agile)

- korostivat itse ohjelmistoa sekä ohjelmiston asiakkaan ja toteuttajien merkitystä yksityiskohtaisen suunnittelun ja dokumentaation sijaan

Ketterien menetelmien synty

Ristiriita/turhauma synnytti joukon **ketteriä prosessimalleja** (agile)

- korostivat itse ohjelmistoa sekä ohjelmiston asiakkaan ja toteuttajien merkitystä yksityiskohtaisen suunnittelun ja dokumentaation sijaan
- Useita samanhenkisiä menetelmiä 90-luvun loppupuolelta lähtien

Ketterä manifesti 2001

Ketterä manifesti 2001



Manifesto for Agile Software Development

We are uncovering better ways of developing
software by doing it and helping others do it.
Through this work we have come to value:

Individuals and interactions over processes and tools
Working software over comprehensive documentation
Customer collaboration over contract negotiation
Responding to change over following a plan

That is, while there is value in the items on
the right, we value the items on the left more.

Kent Beck
Mike Beedle
Arie van Bennekum
Alistair Cockburn
Ward Cunningham
Martin Fowler
James Grenning
Jim Highsmith
Andrew Hunt
Ron Jeffries
Jon Kern
Brian Marick
Robert C. Martin
Steve Mellor
Ken Schwaber
Jeff Sutherland
Dave Thomas

Ketterä manifesti 2001

We are uncovering better ways of developing software by doing it and helping others do it.

Through this work we have come to value:

- **Individuals and interactions** over processes and tools
- **Working software** over comprehensive documentation
- **Customer collaboration** over contract negotiation
- **Responding to change** over following a plan

That is, while there is value in the items on the right, we value the items on the left more

Ketterä manifesti 2001

We are uncovering better ways of developing software by doing it and helping others do it.

Through this work we have come to value:

- **Individuals and interactions** over processes and tools
- **Working software** over comprehensive documentation
- **Customer collaboration** over contract negotiation
- **Responding to change** over following a plan

That is, while there is value in the items on the right, we value the items on the left more

Manifestin laativat ja allekirjoittivat 17 ketterien menetelmien varhaista pioneeria, mm. Kent Beck, Robert Martin, Ken Schwaber ja Martin Fowler

Ketterät periaatteet, osa 1

Ketterät periaatteet, osa 1

Our highest priority is to satisfy the customer through **early and continuous delivery of valuable software**

Ketterät periaatteet, osa 1

Our highest priority is to satisfy the customer through **early and continuous delivery of valuable software**

Deliver working software frequently, from a couple of weeks to a couple of months, with a preference to the shorter timescale

Ketterät periaatteet, osa 1

Our highest priority is to satisfy the customer through **early and continuous delivery of valuable software**

Deliver working software frequently, from a couple of weeks to a couple of months, with a preference to the shorter timescale

Business people and developers must **work together daily** throughout the project

Ketterät periaatteet, osa 1

Our highest priority is to satisfy the customer through **early and continuous delivery of valuable software**

Deliver working software frequently, from a couple of weeks to a couple of months, with a preference to the shorter timescale

Business people and developers must **work together daily** throughout the project

The most efficient and effective method of conveying information to and within a development team is **face-to-face** conversation

Ketterät periaatteet, osa 1

Our highest priority is to satisfy the customer through **early and continuous delivery of valuable software**

Deliver working software frequently, from a couple of weeks to a couple of months, with a preference to the shorter timescale

Business people and developers must **work together daily** throughout the project

The most efficient and effective method of conveying information to and within a development team is **face-to-face** conversation

Welcome changing requirements, even late in development. Agile processes harness change for the customer's competitive advantage

Ketterät periaatteet, osa 2

Build projects around motivated individuals. Give them the environment and support they need, and **trust them to get the job done**

Ketterät periaatteet, osa 2

Build projects around motivated individuals. Give them the environment and support they need, and **trust them to get the job done**

The best architectures, requirements, and designs emerge from **self-organizing teams**

Ketterät periaatteet, osa 2

Build projects around motivated individuals. Give them the environment and support they need, and **trust them to get the job done**

The best architectures, requirements, and designs emerge from **self-organizing teams**

At regular intervals, the **team reflects on how to become more effective**, then tunes and adjusts its behavior accordingly

Ketterät periaatteet, osa 2

Build projects around motivated individuals. Give them the environment and support they need, and **trust them to get the job done**

The best architectures, requirements, and designs emerge from **self-organizing teams**

At regular intervals, the **team reflects on how to become more effective**, then tunes and adjusts its behavior accordingly

Simplicity – the art of maximizing the amount of work not done – is essential

Ketterät periaatteet, osa 2

Build projects around motivated individuals. Give them the environment and support they need, and **trust them to get the job done**

The best architectures, requirements, and designs emerge from **self-organizing teams**

At regular intervals, the **team reflects on how to become more effective**, then tunes and adjusts its behavior accordingly

Simplicity – the art of maximizing the amount of work not done – is essential

Continuous attention to **technical excellence and good design** enhances agility

Ketterät periaatteet, osa 2

Build projects around motivated individuals. Give them the environment and support they need, and **trust them to get the job done**

The best architectures, requirements, and designs emerge from **self-organizing teams**

At regular intervals, the **team reflects on how to become more effective**, then tunes and adjusts its behavior accordingly

Simplicity – the art of maximizing the amount of work not done – is essential

Continuous attention to **technical excellence and good design** enhances agility

Agile processes promote **sustainable development**. The sponsors, developers, and users should be able to maintain a constant pace indefinitely

Ketterät menetelmät

Sateenvarjotermi useille ketterille prosessimalleille

- Näistä tunnetuimpia Scrum ja eXtreme programming eli XP
- Molempiin, erityisesti Scrumiin tutustutaan kurssin aikana

Ketterät menetelmät

Sateenvarjotermi useille ketterille prosessimalleille

- Näistä tunnetuimpia Scrum ja eXtreme programming eli XP
- Molempiin, erityisesti Scrumiin tutustutaan kurssin aikana

Ketterä ohjelmistotuotanto on ottanut vaikutteita myös Toyota production systemin taustalla olevasta *lean-ajattelusta*

Leanista peräisin olevia menetelmiä

- Kanban
- Scrumban (Scrumin ja Kanbanin yhdistelmä)

Ketterät menetelmät

Sateenvarjotermi useille ketterille prosessimalleille

- Näistä tunnetuimpia Scrum ja eXtreme programming eli XP
- Molempiin, erityisesti Scrumiin tutustutaan kurssin aikana

Ketterä ohjelmistotuotanto on ottanut vaikutteita myös Toyota production systemin taustalla olevasta *lean-ajattelusta*

Leanista peräisin olevia menetelmiä

- Kanban
- Scrumban (Scrumin ja Kanbanin yhdistelmä)

... sekä laajan mittakaavan leaniin ja ketterään kehitykseen tarkoitettuja menetelmiä kuten SAFe ja LeSS

Myös nämä kuuluvat kurssin aihepiiriin

Viikon 1 laskarit

Viikon 1 laskarit

Ohjelmistokehityksen käytännön työkaluja

- Versionhallinta: Git
- Automatisoitu testaus: Unittest
- Projektin riippuvuuksienhallinta ja “buildaus”: uv
- CI- ja build-palvelinohjelmisto: GitHub Actions

Tavoitteena mahdollistaa hallittu ja toistettavissa oleva ohjelmistokehitys

Viikon 1 laskarit

Ohjelmistokehityksen käytännön työkaluja

- Versionhallinta: Git
- Automatisoitu testaus: Unittest
- Projektin riippuvuuksienhallinta ja “buildaus”: uv
- CI- ja build-palvelinohjelmisto: GitHub Actions

Tavoitteena mahdollistaa hallittu ja toistettavissa oleva ohjelmistokehitys

Deadline maanantaina klo 23.59

Generatiivinen tekoäly

- Loppuuko ohjelmistokehitys?

Generatiivinen tekoäly

- Loppuuko ohjelmistokehitys?
- Tuskin, mutta se muuttuu

Generatiivinen tekoäly

- Loppuuko ohjelmistokehitys?
- Tuskin, mutta se muuttuu
- Opettele perusteet syvällisesti
  - agentin tuotosta ei pysty arvioimaan eikä agenttia ohjaamaan, jos ei itse ymmärrä mitä koodi tekee
  - ihmistä tarvitaan näyttämään suunta: mitä asiakas oikeasti tarvitsee

Generatiivinen tekoäly

- Loppuuko ohjelmistokehitys?
- Tuskin, mutta se muuttuu
- Opettele perusteet syvällisesti
  - agentin tuotosta ei pysty arvioimaan eikä agenttia ohjaamaan, jos ei itse ymmärrä mitä koodi tekee
  - ihmistä tarvitaan näyttämään suunta: mitä asiakas oikeasti tarvitsee
- Agentit: esim. Copilotin Agent-tila, Claude Code, Cursor

Generatiivinen tekoäly

- Loppuuko ohjelmistokehitys?
- Tuskin, mutta se muuttuu
- Opettele perusteet syvällisesti
  - agentin tuotosta ei pysty arvioimaan eikä agenttia ohjaamaan, jos ei itse ymmärrä mitä koodi tekee
  - ihmistä tarvitaan näyttämään suunta: mitä asiakas oikeasti tarvitsee
- Agentit: esim. Copilotin Agent-tila, Claude Code, Cursor
- Tekoäly toimii *vahvistimena* (DORA 2025): parantaa hyvin toimivia tiimejä, pahentaa heikosti toimivien ongelmia

Generatiivinen tekoäly

- Loppuuko ohjelmistokehitys?
- Tuskin, mutta se muuttuu
- Opettele perusteet syvällisesti
  - agentin tuotosta ei pysty arvioimaan eikä agenttia ohjaamaan, jos ei itse ymmärrä mitä koodi tekee
  - ihmistä tarvitaan näyttämään suunta: mitä asiakas oikeasti tarvitsee
- Agentit: esim. Copilotin Agent-tila, Claude Code, Cursor
- Tekoäly toimii *vahvistimena* (DORA 2025): parantaa hyvin toimivia tiimejä, pahentaa heikosti toimivien ongelmia
- Suosittelen hyödyntämään kunhan vaan tietää mitä tekee
  - Vibe coding vs. agentic engineering
  - Tarkemmin: materiaali Tekoäly ohjelmistotuotannossa